



Последовательности: строки

Докладчик: Евграфов Михаил



Строки: обзор

Характеристики строки

гомогенная

ограниченная

упорядоченная

итерируемая

неизменяемая

Гомогенность

```
word = "python"  
first_char = word[0]
```

```
print(first_char)           # p  
print(type(first_char))    # <class 'str'>  
print(len(first_char))     # 1
```

Ограниченность

```
print(len("python"))
```

```
# 6
```

```
print(len("hello, world"))
```

```
# 12 - пробел тоже считаются символами
```

```
print(len(""))
```

```
# 0
```

Упорядоченность

```
word = "python"
```

```
print(word[0])    # p -- первый символ  
print(word[2])    # t  
print(word[-1])   # n -- последний символ
```

Итерируемость

```
for char in "abc":  
    print(char)
```

```
# a
```

```
# b
```

```
# c
```

Неизменяемость

```
word = "python"
```

```
word[0] = "P"
```

```
# TypeError: 'str' object does not support ...
```

Где применяются строки

ВВОД И ВЫВОД

имена и пароли

пути к файлам

JSON, CSV, HTML

**команды и
запросы**



str() и repr()

str()

```
print(repr(str()))    # ''
print(str(42))         # 42
print(str(3.14))       # 3.14
print(str(True))       # True
print(str(None))       # None
```

str() для составных объектов

```
print(str([1, 2, 3])) # [1, 2, 3]
```

```
print(str((1, "a"))) # (1, 'a')
```

```
print(type(str([1, 2, 3])))
```

```
# <class 'str'> - одна строка, а не список
```

repr()

```
print(42)           # 42
```

```
print("42")         # 42
```

```
print(repr(42))     # 42
```

```
print(repr("42"))   # '42'
```

repr() и служебные символы

```
text = "hello \n"
```

```
print(text)
```

```
# hello - перевод строки не виден
```

```
print(repr(text))
```

```
# 'hello \n'
```

str vs repr

```
from decimal import Decimal

number = Decimal("3.14")
print(
    f"number as str: {str(number)}",
    f"number as repr: {repr(number)}",
    sep="\n",
)
# number as str: 3.14
# number as repr: Decimal('3.14')
```

Что выбрать?

`str(obj)`

для человека

использует `print()`

`repr(obj)`

для разработчика

REPL и отладка



Строковые литералы

Одинарные и двойные кавычки

```
single_quoted = 'hello'  
double_quoted = "hello"
```

```
print(single_quoted == double_quoted) # True  
print(type(single_quoted))           # <class 'str'>  
print(type(double_quoted))           # <class 'str'>
```

Кавычки внутри текста

```
print("It's a sunny day")  
# It's a sunny day
```

```
print('He said: "Hello"')  
# He said: "Hello"
```

```
print('It\'s "quoted"')  
# It's "quoted"
```

Пустая строка

```
empty_string = ""  
  
print(len(empty_string))  
# 0  
print(bool(empty_string))  
# False  
print(bool(" "))  
# True
```

Литерал — одна строка

```
message = "first line  
second line"
```

```
# SyntaxError: unterminated string literal...
```

Тройные кавычки

```
poem = """Roses are red,  
Violets are blue."""  
  
print(poem)  
# Roses are red,  
# Violets are blue.  
  
print(repr(poem))  
# 'Roses are red,\nViolets are blue.'
```

Сохранение табуляции

```
text = """  
first  
second  
"""
```

```
print(repr(text))  
# '\nfirst\nsecond\n'
```

Экранирование перевода строки

```
text = """\nfirst\nsecond"""
```

```
print(repr(text)) # 'first\nsecond'
```

Неявная конкатенация

```
text = "hello, " "world"  
print(text)  # hello, world
```

Разбиение

```
long_message = (  
    "This is a very long message, "  
    "which is split into several lines "  
    "to keep the code readable."  
)  
  
print(long_message)  
# This is a very long message ...
```

Пропущенная запятая

```
fruits = [  
    "apple",  
    "banana"  
    "cherry",  
]
```

```
print(fruits)           # ['apple', 'bananacherry']  
print(len(fruits))      # 2
```



Escape-символы

Escape-последовательности

`\n`

перевод строки

`\t`

табуляция

`\\`

обратный слеш

`\'`

одинарная
кавычка

`\"`

двойная кавычка

`\v`

верт. табуляция

`\b`

backspace

`\r`

возврат каретки

Один символ в строке

```
print(len("\n"))      # 1
print(len("\\"))       # 1
print(len("a\tb"))     # 3
```

Основные escape-символы

```
print("first\nsecond")
# first
# second
print("Alice\t30")
# Alice    30
print('It\'s a sunny day')
# It's a sunny day
print("He said: \"Hello\"")
# He said: "Hello"
```

Следствие

```
print("C:\new\table")
```

```
print(path)
```

```
# C:
```

```
# ew      able
```

\\ — обратный слеш

```
path = "C:\\Users\\Alice\\notes.txt"
```

```
print(path)
```

```
# C:\Users\Alice\notes.txt
```



r-, f- и t-строки

r-строка

```
regular_path = "C:\\new\\table"  
raw_path = r"C:\new\table"  
  
print(raw_path)  
# C:\new\table  
print(regular_path == raw_path)  
# True - это одна и та же строка  
print(len(r"\n"))  
# 2
```

Ограничения r-строк

```
print(r"say \"hi\"")  
# say \"hi\"
```

```
print(r"C:\folder\")  
# SyntaxError: unterminated string literal
```

Склейка через +

```
name = "Alice"  
age = 30
```

```
print("Name: " + name + ", age: " + str(age))  
# Name: Alice, age: 30
```

f-строка

```
name = "Alice"
```

```
age = 30
```

```
print(f"Name: {name}, age: {age}")
```

```
# Name: Alice, age: 30
```

Выражения внутри {}

```
price = 120
quantity = 3
name = "alice"

print(f"Total: {price * quantity}")
# Total: 360
print(f"Name: {name.upper()}")
# Name: ALICE
print(f"Even: {quantity % 2 == 0}")
# Even: False
print(f"Status: {'ok' if price > 100 else 'low'}")
# Status: ok
```

Фигурные скобки как символы

```
value = 5
```

```
print(f"Set: {{{value}}}")
```

```
# Set: {5}
```

```
print(f"{{value}}")
```

```
# {value} - скобки удвоены, подстановки нет
```

Отладочный вывод {value=}

```
counter = 7  
text = "hi"
```

```
print(f"{counter=}")           # counter=7  
print(f"{text=}")             # text='hi'  
print(f"{counter * 2 =}")      # counter * 2 = 14
```

Флаг !r

```
word = "hello\n"
```

```
print(f"{word!r}")
```

```
# 'hello\n'
```

t-строка (Python 3.14)

```
# Python 3.14+  
name = "Alice"  
  
greeting = t"Hello, {name}!"  
  
print(type(greeting))  
# <class 'string.templatelib.Template'>
```



Строки и последовательности

len()

```
print(len("python"))
```

```
# 6
```

```
print(len(""))
```

```
# 0
```

```
print(len("a b"))
```

```
# 3 - пробел тоже символ
```

in и not in

```
text = "hello, world"
```

```
print("h" in text)
```

```
# True - СИМВОЛ
```

```
print("world" in text)
```

```
# True - подстрока
```

```
print("hello world" in text)
```

```
# False - нет такой подстроки
```

```
print("planet" not in text)
```

```
# True
```

Конкатенация +

```
first_part = "Hello, "  
second_part = "world"
```

```
print(first_part + second_part)  
# Hello, world
```

Оператор +=

```
text = "abc"  
identity_before = id(text)
```

```
text += "def"
```

```
print(text) # abcdef  
print(id(text) == identity_before) # False
```

Повторение *

```
print("ab" * 3)      # ababab
print("-" * 20)      # -----
print("ab" * 0)      # (пустая строка)
print("ab" * -2)     # (пустая строка)
```

Индексы

```
word = "python"
```

```
print(word[0])          # p
```

```
print(word[-1])         # n
```

```
print(word[len(word) - 1]) # n
```

```
print(word[10])
```

```
# IndexError: string index out of range
```

Присваивание по индексу

```
word = "python"
```

```
word[0] = "P"
```

```
# TypeError: 'str' object does not support...
```

Обход в цикле

```
for char in "abc":  
    print(char)
```

```
# a
```

```
# b
```

```
# c
```



Сравнение и логика

Равенство == и !=

```
print("python" == "python")
```

```
# True
```

```
print("python" == "Python")
```

```
# False - регистр важен
```

```
print("python" == "python ")
```

```
# False - пробел в конце тоже символ
```

```
print("abc" != "abd")
```

```
# True
```

Сравнение строк



Лексикографический порядок

```
print("apple" < "banana")
# True   - 'a' (97) < 'b' (98)
print("apple" < "apricot")
# True   - 'p' (112) < 'r' (114)
print("app" < "apple")
# True   - "app" - префикс, а значит короче
print("" < "a")
# True
print("abc" <= "abc")
# True
```

Сравнение по кодам

"Zebra"

<

"apple"

"Z" = 90

"a" = 97

90 < 97 # True

Истинность строк

```
print(bool(""))           # False
print(bool("a"))          # True
print(bool(" "))          # True
print(bool("0"))          # True
print(bool("False"))      # True
```

Логический контекст

```
user_input = ""
```

```
if user_input:  
    print("Got input")
```

```
else:  
    print("Empty input")  
# Empty input
```

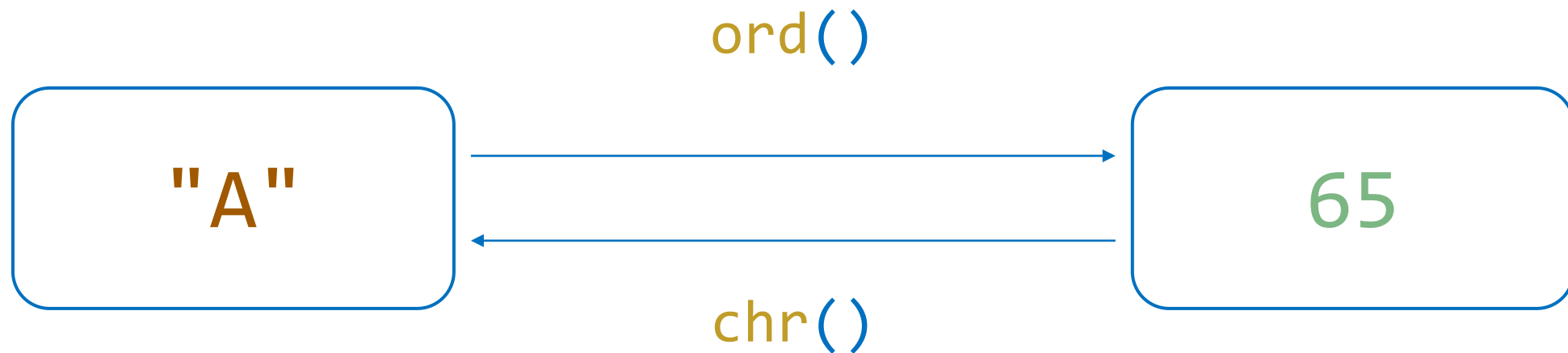
Значение по умолчанию

```
name = ""  
display_name = name or "anonymous"  
  
print(display_name)           # anonymous  
print("hello" and "world")    # world  
print("" and "world")         # (пустая строка)
```



Работа с кодировкой

ord() и chr()



СИМВОЛ ↔ кодовая позиция Unicode

ord()

```
print(ord("A"))    # 65  
print(ord("a"))    # 97  
print(ord("0"))    # 48  
print(ord("€"))    # 8364
```

```
print(ord("ab"))  
# TypeError: ord() expected a character...
```

chr()

<code>print(chr(65))</code>	<code># A</code>
<code>print(chr(97))</code>	<code># a</code>
<code>print(chr(8364))</code>	<code># €</code>
<code>print(chr(0x20ac))</code>	<code># €</code>

chr(): ValueError

```
print(chr(-1))
```

```
# ValueError: chr() arg not in range(0x110000)
```

```
print(chr(0x110000))
```

```
# ValueError: chr() arg not in range(0x110000)
```

ord() и chr() обратны

```
print(chr(ord("Z")))
```

```
# Z
```

```
print(ord(chr(1234)))
```

```
# 1234
```



Способы форматирования

Три способа форматирования

`f"{name}"`
по умолчанию

`"{}".format(name)`
шаблон отдельно от
данных

`"%s" % name`
старый код и logging

f-строки

```
name = "Alice"
```

```
age = 30
```

```
print(f"{name} is {age} years old")
```

```
# Alice is 30 years old
```

```
print(f"Next year {name} will be {age + 1}")
```

```
# Next year Alice will be 31
```

format()

```
print("{} is {} years old".format("Alice", 30))  
# Alice is 30 years old
```

```
print("{1} is {0} years old".format(30, "Bob"))  
# Bob is 30 years old
```

```
print("{name} is {age} years old".format(name="Carol", age=25))  
# Carol is 25 years old
```

```
print("{0} {0} {0}".format("echo"))  
# echo echo echo
```

format(): индексы

```
point = (3, 4)
```

```
names = ["Dave", "Eve"]
```

```
print("x={0[0]}, y={0[1]}".format(point))
```

```
# x=3, y=4
```

```
print("Hi, {0[1]}!".format(names))
```

```
# Hi, Eve!
```

format(): шаблон отдельно

```
GREETING_TEMPLATE = (  
    "Hello, {name}! You have {messages_count} new messages."  
)  
  
print(GREETING_TEMPLATE.format(name="Alice", messages_count=5))  
# Hello, Alice! You have 5 new messages.  
  
print(GREETING_TEMPLATE.format(name="Bob", messages_count=0))  
# Hello, Bob! You have 0 new messages.
```

Оператор %

```
name = "Alice"
```

```
age = 30
```

```
score = 92.456
```

```
print("%s is %d years old" % (name, age))
```

```
# Alice is 30 years old
```

```
print("Score: %f" % score)
```

```
# Score: 92.456000
```

```
print("Progress: %d%%" % 75)
```

```
# Progress: 75%
```

Ловушка с кортежем

```
point = (1, 2)
```

```
print("Point: %s" % point)
```

```
# TypeError: not all arguments converted...
```

```
print("Point: %s" % (point,))
```

```
# Point: (1, 2)
```



Форматирование чисел

Точность: тип f

```
pi = 3.14159265
```

```
print(f"{pi:.0f}") # 3
print(f"{pi:.2f}") # 3.14
print(f"{pi:.4f}") # 3.1416
print(f"{pi:f}")   # 3.141593
print(f"{5:.2f}")  # 5.00
```

Типы e и g

```
big_number = 123456.789
```

```
print(f"{big_number:.2e}")    # 1.23e+05  
print(f"{big_number:.3g}")    # 1.23e+05  
print(f"{3.14159:.3g}")       # 3.14  
print(f"{0.5:.3g}")           # 0.5
```

Системы счисления

```
number = 255
print(f"{number:d}")      # 255
print(f"{number:b}")      # 11111111
print(f"{number:o}")      # 377
print(f"{number:x}")      # ff
print(f"{number:X}")      # FF
print(f"{number:#x}")     # 0xff
print(f"{number:#b}")     # 0b11111111
```

Проценты

```
share = 0.256
```

<pre>print(f"{share:%}")</pre>	<pre># 25.600000%</pre>
<pre>print(f"{share:.1%}")</pre>	<pre># 25.6%</pre>
<pre>print(f"{share:.0%}")</pre>	<pre># 26%</pre>
<pre>print(f"{1:.0%}")</pre>	<pre># 100%</pre>

Разделители разрядов

```
population = 1234567
```

```
print(f"{population:,}")          # 1,234,567
print(f"{population:_}")          # 1_234_567
print(f"{1234567.891:,.2f}")      # 1,234,567.89
```

Ширина и нули слева

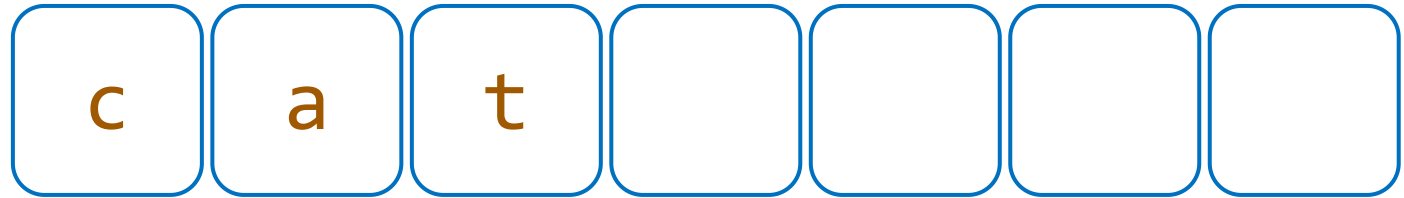
```
print(f"[{42:6d}]")           # [    42]
print(f"[{42:06d}]")          # [000042]
print(f"[{-42:06d}]")         # [-00042]
print(f"[{3.14159:08.3f}]")   # [0003.142]
```



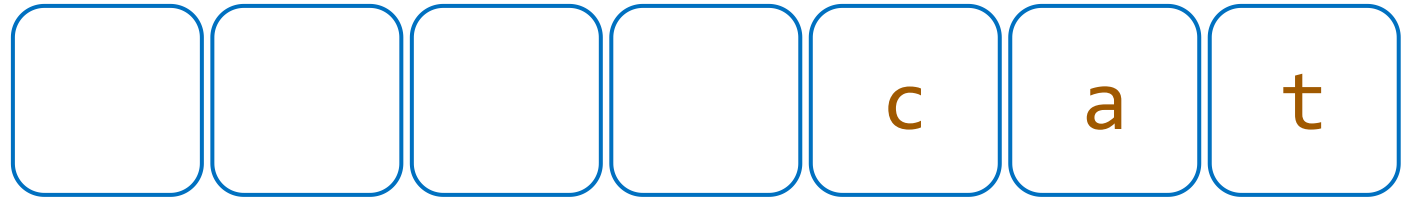
Выравнивание строк

ljust(), rjust(), center()

word.ljust(7)



word.rjust(7)



word.center(7)



ljust(), rjust(), center()

```
word = "cat"
```

```
print(f"[{word.ljust(7)}]")          # [cat   ]
print(f"[{word.rjust(7)}]")          # [      cat]
print(f"[{word.ljust(7, ' ')}]")      # [cat...]
print(f"[{word.rjust(7, '*')}]")      # [****cat]
print(f"[{word.center(7, '-')}]")    # [--cat--]
```

Выравнивание < > ^

```
word = "cat"
```

```
print(f"[{word:<7}]") # [cat   ]
print(f"[{word:>7}]") # [      cat]
print(f"[{word:^7}]") # [   cat   ]
print(f"[{word:7}]")  # [cat     ]
print(f"[{42:7}]")    # [      42]
```

Символ-заполнитель

```
print(f"[{word:.<7}]")    # [cat....]
print(f"[{word:*>7}]")   # [****cat]
print(f"[{word:-^7}]")   # [--cat--]
```

Выравнивание и число

```
pi = 3.14159265
```

```
print(f"[{pi:.2f}]")      # [3.14]
print(f"[{pi:10.2f}]")    # [          3.14]
print(f"[{pi:<10.2f}]")   # [3.14          ]
print(f"[{pi:^10.2f}]")   # [      3.14      ]
print(f"[{pi:*>10.2f}]")  # [*****3.14]
```

Устройство спецификатора

`f"{pi:*>10.2f}"`

*

заполнитель

>

выравнивание

10

ширина

.2

точность

f

тип



Изменение регистра

lower() и upper()

```
text = "Hello, World 123"
```

```
print(text.lower()) # hello, world 123  
print(text.upper()) # HELLO, WORLD 123  
print(text)         # Hello, World 123
```

capitalize()

```
print("hello world".capitalize())  
# Hello world
```

```
print("hELLO wORLD".capitalize())  
# Hello world
```

```
print("python and NASA".capitalize())  
# Python and nasa
```

title()

```
print("the quick brown fox".title())  
# The Quick Brown Fox
```

```
print("hello-world".title())  
# Hello-World
```

```
print("they're here".title())  
# They'Re Here
```

```
print("3rd place".title())  
# 3Rd Place
```



Проверка и модуль `string`

Методы проверки

`islower()`

все буквы строчные

`isupper()`

все буквы заглавные

`isalpha()`

все — буквы

`isdigit()`

все — цифры

`isalnum()`

буквы или цифры

`isspace()`

только пробельные

islower() и isupper()

```
print("hello".islower())      # True
print("hello 123".islower())  # True
print("Hello".islower())      # False
print("123".islower())        # False
```

```
print("HELLO".isupper())      # True
print("HELLO world".isupper()) # False
```

isalpha(), isdigit()

```
print("hello".isalpha())      # True
print("hello world".isalpha()) # False
print("hello1".isalpha())     # False
```

```
print("12345".isdigit())      # True
print("-5".isdigit())         # False
print("3.14".isdigit())       # False
```

```
print("abc123".isalnum())     # True
print("abc_123".isalnum())    # False
```

isalpha() и Unicode

```
print("é".isalpha())  
# True
```

```
print("\u03c0".isalpha())  
# True - греческая буква "пи"
```

isspace()

```
print("    ".isspace())    # True
print("\t\n".isspace())    # True
print(" a ".isspace())     # False
```

Пустая строка

```
print("").isalpha()) # False  
print("").isdigit()) # False  
print("").isspace()) # False
```

Константы string

```
import string

print(string.ascii_lowercase)
# abcdefghijklmnopqrstuvwxyz
print(string.digits)
# 0123456789
print(string.punctuation)
# !"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
print(repr(string.whitespace))
# ' \t\n\r\x0b\x0c'
```

Практические примеры

```
print("é".isalpha())           # True
print("é" in string.ascii_letters) # False
```

```
print("!" in string.punctuation) # True
print("a" in string.punctuation) # False
```



Очистка строк

Методы очистки

`strip()`
символы с краев

`replace()`
подстрока везде

`removeprefix()`
точный префикс /
суффикс

Зачем очищать строки

```
answer = "  yes \n"
```

```
print(answer == "yes")           # False
```

```
print(answer.strip() == "yes")   # True
```

strip(), lstrip(), rstrip()

```
text = " \t hello  world \n"
```

```
print(repr(text.strip()))      # 'hello  world'  
print(repr(text.lstrip()))    # 'hello  world \n'  
print(repr(text.rstrip()))    # ' \t hello  world'
```

strip(): набор символов

```
print("***hello***".strip("*"))    # hello
print("xyxhelloyx".strip("xy"))    # hello
print("[1, 2, 3]".strip("[]"))     # 1, 2, 3
print("line\n".rstrip("\n"))        # line

print("happy.py".rstrip(".py"))     #
```

replace()

```
text = "one fish, two fish, red fish"
print(text.replace("fish", "cat"))
# one cat, two cat, red cat
print(text.replace("fish", "cat", 1))
# one cat, two fish, red fish
```

```
phone = "+7 (999) 123-45-67"
print(phone.replace(" ", ""))
# +7(999)123-45-67
print(phone.replace(" ", "").replace("-", ""))
# +7(999)1234567
```

removeprefix() и removesuffix()

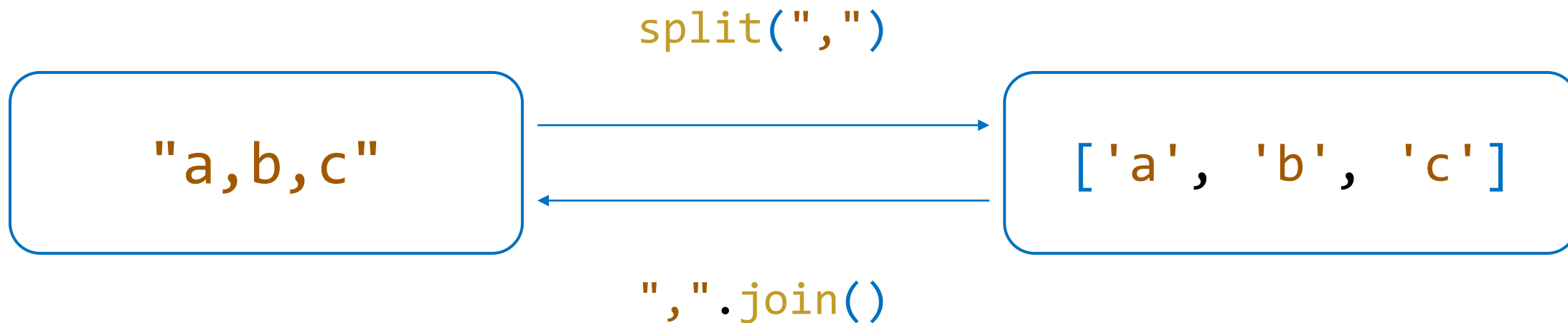
```
file_name = "report.txt"  
url = "https://example.com"
```

```
print(file_name.removesuffix(".txt"))    # report  
print(file_name.removesuffix(".csv"))    # report.txt  
print(url.removeprefix("https://"))     # example.com
```



split() и join()

split() и join()



строка ↔ список строк

split(): без аргументов

```
sentence = " the quick\tbrown\nfox "
```

```
print(sentence.split())  
# ['the', 'quick', 'brown', 'fox']
```

split(): sep

```
print("a,b,c".split(","))  
# ['a', 'b', 'c']
```

```
print("a,,b".split(","))  
# ['a', '', 'b'] - пустой элемент сохранен
```

```
print("a--b--c".split("--"))  
# ['a', 'b', 'c']
```

```
print("2024-05-17".split("-"))  
# ['2024', '05', '17']
```

split(): maxsplit

```
line = "key=value=extra"
```

```
print(line.split("=", 1))  
# ['key', 'value=extra']
```

```
print(line.split("=", 2))  
# ['key', 'value', 'extra']
```

rsplit()

```
print("a.b.c.d".split(".", 1))  
# ['a', 'b.c.d']
```

```
print("a.b.c.d".rsplit(".", 1))  
# ['a.b.c', 'd']
```

join()

```
words = ["to", "be", "or", "not", "to", "be"]
```

```
print(" ".join(words))      # to be or not to be
print(", ".join(words))     # to, be, or, not, to, be
print("").join(words)       # tobeornottobe
print("-".join(["a"]))      # a
print(" ".join([]))         # (пустая строка)
```

join(): только строки

```
numbers = [1, 2, 3]
```

```
print(", ".join(numbers))
```

```
# TypeError: sequence item 0: expected str...
```

```
print(", ".join([str(number) for number in numbers]))
```

```
# 1, 2, 3
```

join() вместо +=

```
parts = []
```

```
for number in range(1, 6):  
    parts.append(str(number))
```

```
print(", ".join(parts))    # 1,2,3,4,5
```



Поиск в строках

startswith(), endswith()

```
file_name = "report_2024.txt"
```

```
print(file_name.startswith("report")) # True
print(file_name.startswith("Report")) # False
print(file_name.endswith(".txt"))     # True
print(file_name.endswith(".csv"))     # False
```

Кортеж префиксов

```
url = "https://example.com"
image = "photo.PNG"

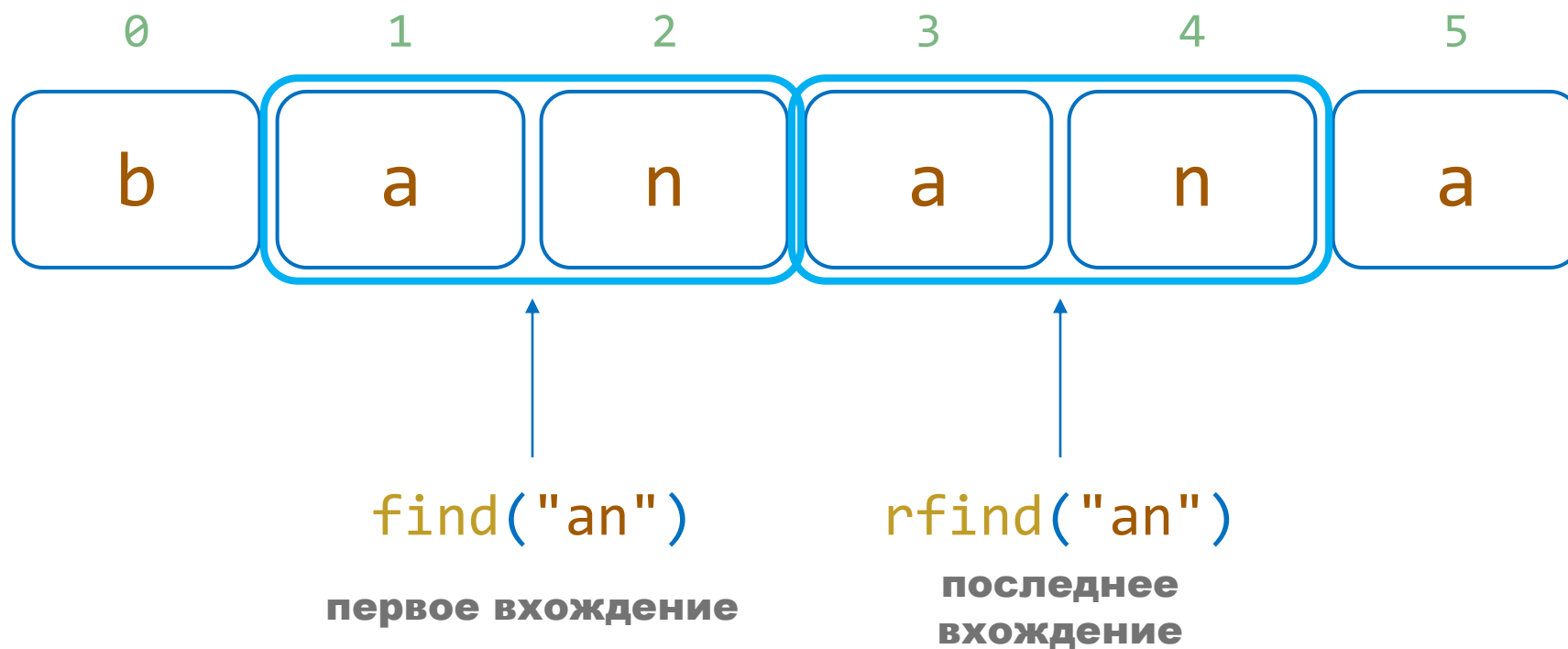
print(url.startswith(("http://", "https://")))
# True
print(image.lower().endswith((".png", ".jpg", ".gif")))
# True
```

find() и rfind()

```
text = "banana"
```

```
print(text.find("an"))    # 1 - первое вхождение
print(text.rfind("an"))   # 3 - последнее вхождение
print(text.find("a"))     # 1
print(text.rfind("a"))    # 5
print(text.find("xyz"))   # -1 - не найдено
```

find() и rfind()



find(): start и end

```
text = "banana"
```

```
print(text.find("an", 2))
```

```
# 3 - поиск начинается с индекса 2
```

```
print(text.find("an", 2, 4))
```

```
# -1 - в срезе text[2:4] == "na" подстроки нет
```

`index()` и `rindex()`

```
text = "banana"
```

```
print(text.index("an"))    # 1
```

```
print(text.rindex("an"))   # 3
```

```
print(text.index("xyz"))
```

```
# ValueError: substring not found
```



Спасибо за внимание!